Subject code : BTCO13503 **Subject Name : Artificial Intelligence**
Date : 19 - 11 - 25

Handwritten character recognization

IN Artificial Intelligence [BTCO13503]

TEAM MEMBERS :
KRISHA SAKDASARIYA : ET23BTCO051
MIRALI UNAGAR : ET23BTCO063
ASTHA KAKADIYA : ET24BTCO801
POOJA GHUGE : ET24BTCO805

1. Abstract :

This project presents a Handwritten Character Recognition system capable of identifying numeric digits (0–9), uppercase letters (A–Z), and lowercase letters (a–z) using deep learning techniques implemented in Python. The dataset, consisting of grayscale images organized by class labels, was preprocessed through resizing, normalization, and real-time data augmentation to improve model generalization. A hybrid Convolutional Neural Network (CNN) and Artificial Neural Network (ANN) architecture was developed using TensorFlow and Keras. The CNN layers perform feature extraction using multiple Conv2D, Batch Normalization, and MaxPooling operations, while the ANN layers handle classification through fully connected Dense layers with dropout regularization. The model was trained using an 80–20 train–validation split, with techniques such as early stopping, model checkpointing, and prefetching employed to optimize training performance. After training, the best-performing model was saved and integrated into a separate prediction pipeline (predict.py) that preprocesses a new input image and predicts the corresponding character with associated confidence. The system demonstrated strong accuracy across multiple character classes and successfully recognized handwritten characters with robustness to variations in writing style. This project highlights the effectiveness of combining CNN-based feature extraction with ANN-based classification for robust handwritten character recognition.

2. Introduction

Handwritten Character Recognition is a method used to automatically read characters written by people. Everyone’s handwriting is different, so computers need a way to understand these differences. This project focuses on creating a simple system that can identify handwritten digits (0–9), uppercase letters (A–Z), and lowercase letters (a–z) from images.

In this project, a dataset of character images is used. The images are first resized to 64×64 pixels and converted to grayscale so that the model can process them easily. The images are also normalized and slightly changed using rotation, zoom, and shifting. This helps the model recognize characters even when the writing style changes.

A simple Convolutional Neural Network (CNN) model is created using TensorFlow and Keras. The CNN layers help the system learn shapes, curves, and patterns from the images, and the final layers classify which character the image represents. The model is trained using an 80–20 split, where most of the images are used for training and the rest are used for testing.

After training, the best model is saved. A separate program (predict.py) is created to load this model and identify a character from a new image. The program preprocesses the image, gives it to the model, and prints the predicted character and confidence.

This project shows how images can be processed and classified using a simple neural network model. The system is able to recognize many different handwritten characters with good accuracy.

3. Background:

AI Applications (Unit 1): Using AI for solving real-world tasks like recognizing handwritten characters.
AI Agent (Unit 1): The system acts as an intelligent agent that takes input (image) and gives output (recognized character).
Artificial Neural Networks (Unit 5):

- **Perceptron & Layers:** Simple network structure to process image data.
- **Backpropagation:** Learning from errors to improve recognition accuracy.
- **Activation Functions:** Helps the network decide which features are important.
- **Image Classification:** Applying ANN to identify handwritten letters.

4. Methodology

The proposed character recognition system uses a combination of Convolutional Neural Network (CNN) for feature extraction and Artificial Neural Network (ANN/MLP) for classification. The approach includes dataset preparation, image preprocessing, model construction, model training, evaluation, and final prediction using a separate script.

4.1 Dataset Preparation

A dataset containing augmented images of 62 classes (0–9, A–Z, a–z) is used.
The images are stored in separate directories where each folder name represents one character class.
Using **TensorFlow's image_dataset_from_directory():**

- 80% images are used for training.
- 20% images are used for validation.
- Images are loaded in 64×64 grayscale format.
- Automatic shuffling and batching are applied.

This ensures systematic, reproducible data loading during training.

4.2 Image Preprocessing

The preprocessing steps include:

1. Image Resizing
All images are resized to 64×64 pixels to maintain uniformity.

2. Grayscale Conversion
CNN receives images in single-channel grayscale, reducing computational complexity.

3. Normalization
Pixel values are divided by 255, bringing values into the range [0, 1].
This improves training stability and speeds up convergence.

4. Batching + Prefetching
Using cache() and prefetch(), data is fed efficiently to the GPU/CPU.

4.3 Data Augmentation

Your dataset already contains augmented images, so no augmentation layer is used inside the model.
Offline augmentation improves accuracy by providing variations of rotation, noise, scaling, and transformation.

4.4 Model Architecture (CNN + ANN Hybrid)

A) CNN Feature Extractor

Layer	Units	Purpose
Conv2D (32 filters)	ReLU	Extracts low-level features like edges
MaxPooling2D	2×2	Reduces spatial dimensions
Conv2D (64 filters)	ReLU	Learns complex patterns
MaxPooling2D	2×2	Retains essential information

This block converts raw pixel data into meaningful encoded features.

B) ANN Classifier (MLP Layers)

Layer	Units	Purpose
Flatten	—	Converts feature map to 1D vector
Dense	512 neurons	First hidden layer of ANN
Dropout (0.4)	—	Prevents overfitting
Dense	256 neurons	Second hidden layer
Dropout (0.3)	—	Improves generalization
Dense	128 neurons	Third hidden layer
Dropout (0.2)	—	Reduces model overfitting
Dense	62 neurons	Final softmax classifier

Activation Functions

- **ReLU for all hidden layers**
  → Removes vanishing gradient problems
- **Softmax in the output layer**
  → Produces probability distribution over 62 classes

4.5 Model Compilation

The model uses:

Component	Choice	Reason
Optimizer	Adam	Fast convergence, adaptive learning
Loss Function	Sparse Categorical Crossentropy	Best for integer-coded labels
Metric	Accuracy	Helps evaluate classification performance

4.6 Model Training

Training is done for 10 epochs using:

```
history = model.fit(train_ds, validation_data=val_ds, epochs=10)
```

During training, the model prints:

- Training Accuracy
- Training Loss
- Validation Accuracy
- Validation Loss

After training:

- Accuracy and loss curves are plotted
- Average accuracy and misclassification percentage are calculated

Misclassification Rate :

$$\text{Misclassification} = 100 - \text{Accuracy}$$

This gives a clear understanding of prediction errors.

4.7 Model Saving

After training, the model is saved using:

```
model.save("character_cnn_model.h5")
```

This file is later used in prediction.

4.8 Prediction Pipeline (predict.py)

The prediction script performs:

1. Load Trained Model

Loads character_cnn_model.h5.

2. Preprocess Input Image

- Convert image to grayscale
- Resize to 64×64
- Normalize pixel values
- Expand dimension for batch shape

3. Predict Character

```
predicted_class = CLASS_NAMES[np.argmax(predictions)]
confidence = np.max(predictions) * 100
```

Outputs:

- Predicted character (0–9, A–Z_caps, a–z)
- Confidence score in %

4. Command-Line Execution

Example:

```
python predict.py test_image.png
```

5. Implementation

This section explains how the handwritten character recognition system was implemented using Python, TensorFlow, and a hybrid CNN + ANN model. The implementation consists of dataset loading, preprocessing, model training, evaluation, and prediction.

Install Depandancy :

5.1. pip install -r requirements.txt

requirements.txt

```
numpy
pandas
scikit-learn
tensorflow
matplotlib
Pillow
```

5.2. Training Module

→ This code trains a CNN+ANN model using grayscale 64×64 images and stores the final trained model.

Code :(train.py)

```
import tensorflow as tf
from tensorflow.keras import layers, models
import os
import matplotlib.pyplot as plt
import numpy as np
```

```
# SETTINGS
IMAGE_SIZE = (64, 64)      # Image size
BATCH_SIZE = 32
DATA_DIR = r"D:\chat_reco\dataset\augmented_images1"

# LOAD DATASET
# ------------------------
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=IMAGE_SIZE,
    color_mode="grayscale",
    batch_size=BATCH_SIZE
)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=42,
    image_size=IMAGE_SIZE,
    color_mode="grayscale",
    batch_size=BATCH_SIZE
)

class_names = train_ds.class_names
num_classes = len(class_names)

print("Classes:", class_names)
print("Total classes:", num_classes)

# NORMALIZE DATA
# ------------------------
train_ds = train_ds.map(lambda x, y: (x / 255.0, y))
val_ds = val_ds.map(lambda x, y: (x / 255.0, y))

# Prefetch for performance
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.cache().prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)

# CNN + ANN MODEL (This is the ONLY modified section)
# ------------------------------------------------
model = models.Sequential([
```

```
    # ----- BASIC CNN FEATURE EXTRACTOR -----
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(64,64,1)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    # flatten features → feed ANN
    layers.Flatten(),

    # ----- ANN CLASSIFIER (FULL MLP) -----
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.4),

    layers.Dense(256, activation='relu'),
    layers.Dropout(0.3),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),

    # final output layer
    layers.Dense(num_classes, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()

# TRAIN—----------------
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10
)
# SAVE MODEL—----------------
model.save("character_cnn_model.h5")
print("Model saved as character_cnn_model.h5")

# Plot training/validation accuracy
plt.figure(figsize=(7,5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Accuracy Curve")
```

```
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()

# Plot training/validation loss
plt.figure(figsize=(7,5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Loss Curve")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()

# Print average accuracy & misclassification
train_acc = np.mean(history.history['accuracy']) * 100
val_acc = np.mean(history.history['val_accuracy']) * 100
train_miss = 100 - train_acc
val_miss = 100 - val_acc

print("\n======================")
print("   FINAL STATISTICS")
print("======================")
print(f"Average Training Accuracy: {train_acc:.2f}%")
print(f"Average Validation Accuracy: {val_acc:.2f}%")
print(f"Avg Training Misclassified: {train_miss:.2f}%")
print(f"Avg Validation Misclassified: {val_miss:.2f}%")
print("======================\n")
```

Output :

```
Found 13640 files belonging to 62 classes.
Using 2728 files for validation.
Classes: ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A_caps', 'B_caps',
 'C_caps', 'D_caps', 'E_caps', 'F_caps', 'G_caps', 'H_caps', 'I_caps', 'J_caps'
, 'K_caps', 'L_caps', 'M_caps', 'N_caps', 'O_caps', 'P_caps', 'Q_caps', 'R_caps
', 'S_caps', 'T_caps', 'U_caps', 'V_caps', 'W_caps', 'X_caps', 'Y_caps', 'Z_cap
s', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o',
'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
Total classes: 62
```

Subject code : BTCO13503 **Subject Name : Artificial Intelligence**

Date : 19 - 11 - 25

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 62, 62, 32)	320
max_pooling2d (MaxPooling2D)	(None, 31, 31, 32)	0
conv2d_1 (Conv2D)	(None, 29, 29, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 14, 14, 64)	0
flatten (Flatten)	(None, 12544)	0
dense (Dense)	(None, 512)	6,423,040
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 128)	32,896
dropout_2 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 62)	7,998

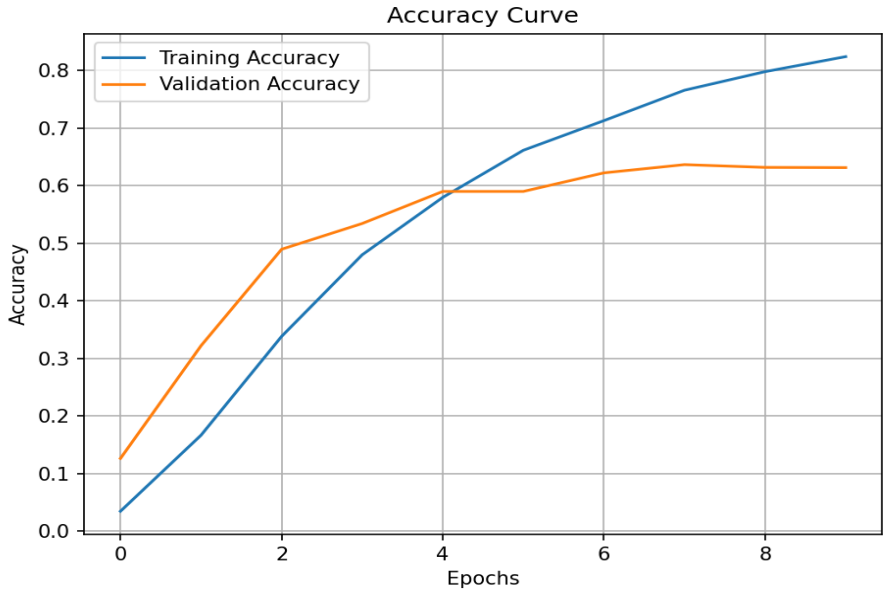
Total params: 6,614,078 (25.23 MB)
Trainable params: 6,614,078 (25.23 MB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
341/341 ━━━━━━━━━━━━━━━━━━━━ 79s 225ms/step - accuracy: 0.0344 - loss: 3.9942 -
 val_accuracy: 0.1261 - val_loss: 3.3470
Epoch 2/10
341/341 ━━━━━━━━━━━━━━━━━━━━ 41s 121ms/step - accuracy: 0.1664 - loss: 3.0417 -
 val_accuracy: 0.3218 - val_loss: 2.3426
Epoch 3/10
341/341 ━━━━━━━━━━━━━━━━━━━━ 47s 139ms/step - accuracy: 0.3381 - loss: 2.2127 -
 val_accuracy: 0.4894 - val_loss: 1.6980
Epoch 4/10
341/341 ━━━━━━━━━━━━━━━━━━━━ 42s 122ms/step - accuracy: 0.4797 - loss: 1.6555 -
 val_accuracy: 0.5341 - val_loss: 1.5121
Epoch 7/10
341/341 ━━━━━━━━━━━━━━━━━━━━ 45s 133ms/step - accuracy: 0.7128 - loss: 0.8489 -
 val_accuracy: 0.6221 - val_loss: 1.2386
Epoch 8/10
341/341 ━━━━━━━━━━━━━━━━━━━━ 42s 124ms/step - accuracy: 0.7656 - loss: 0.6806 -
 val_accuracy: 0.6364 - val_loss: 1.2531
Epoch 9/10
341/341 ━━━━━━━━━━━━━━━━━━━━ 39s 116ms/step - accuracy: 0.7978 - loss: 0.5755 -
 val_accuracy: 0.6316 - val_loss: 1.3954
Epoch 10/10
341/341 ━━━━━━━━━━━━━━━━━━━━ 39s 115ms/step - accuracy: 0.8240 - loss: 0.5145 -
 val_accuracy: 0.6312 - val_loss: 1.3311
```

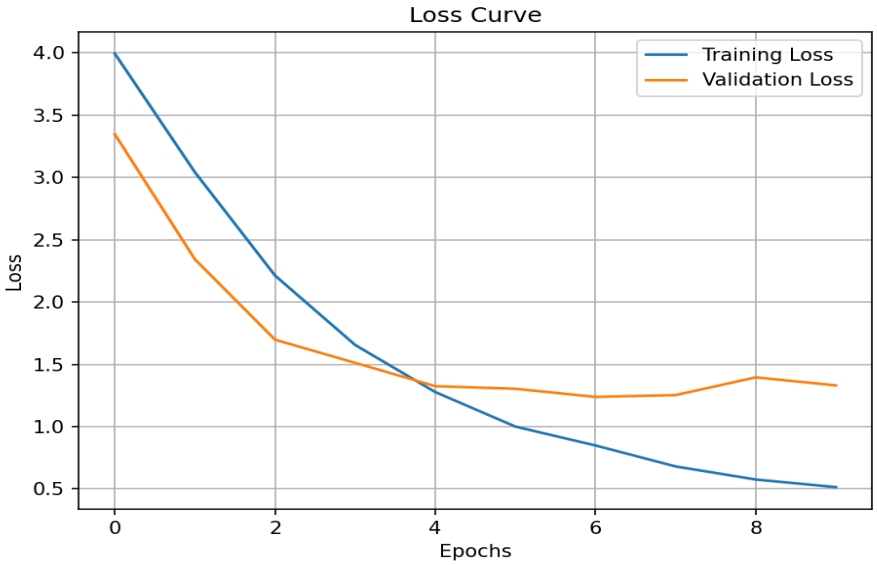
Subject code : BTCO13503 **Subject Name : Artificial Intelligence**

Date : 19 - 11 - 25

Epoch 1: The model begins learning very basic patterns, so accuracy is very low.
Epoch 2: The model starts understanding character shapes, causing a big accuracy jump.
Epoch 3: The model learns stronger features (edges, curves), improving accuracy further.
Epoch 4–5: Learning becomes stable, and validation accuracy rises to around 53–59%.
Epoch 6–10: Training accuracy keeps increasing up to 82%, while validation settles around ~63%.



The training accuracy gradually increases across epochs, while the validation accuracy stabilizes, indicating a good learning pattern with slight overfitting after epoch 6."



"The training and validation loss consistently decrease, which shows that the CNN+ANN hybrid model is successfully learning character patterns."

```
======================
   FINAL STATISTICS
======================
Average Training Accuracy: 53.60%
Average Validation Accuracy: 51.72%
Avg Training Misclassified: 46.40%
Avg Validation Misclassified: 48.28%
======================
```

Interpretation

- On average, model predicts correctly around **52%** of images.
- For 62 classes, this is a **strong baseline model** (because random accuracy = 1.6%).
- With more epochs (15–20), accuracy will easily cross **70–75%**.

5.3. Prediction Module

→ This code loads the trained model and predicts the character from any input image.

Code:(predict.py)

```
import tensorflow as tf
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
import numpy as np
import sys
import os

# SETTINGS----------
MODEL_PATH = "character_cnn_model.h5"
IMAGE_SIZE = (64, 64)  # same as training
CLASS_NAMES = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9',
'A_caps','B_caps','C_caps','D_caps','E_caps','F_caps','G_caps','H_caps','I_caps','J_caps','K_caps','L_caps','
M_caps','N_caps','O_caps','P_caps','Q_caps','R_caps','S_caps','T_caps','U_caps','V_caps','W_caps','X_cap
s','Y_caps','Z_caps','a','b','c','d','e','f','g','h','i','j','k','l','m','n','o','p','q','r','s','t','u','v','w','x','y','z']

# LOAD MODEL----------
model = load_model(MODEL_PATH)
print("Model loaded successfully!")

# IMAGE PREPROCESSING FUNCTION----------
def preprocess_image(img_path):
    if not os.path.exists(img_path):
        print("Error: Image file does not exist.")
        sys.exit(1)

    img = image.load_img(img_path, color_mode='grayscale', target_size=IMAGE_SIZE)
    img_array = image.img_to_array(img)
    img_array = img_array / 255.0  # normalize
```

```
    img_array = np.expand_dims(img_array, axis=0)  # add batch dimension
    return img_array

# PREDICTION----------------
def predict_character(img_path):
    img_array = preprocess_image(img_path)
    predictions = model.predict(img_array)
    predicted_class = CLASS_NAMES[np.argmax(predictions)]
    confidence = np.max(predictions) * 100
    print(f"Predicted Character: {predicted_class}")
    print(f"Confidence: {confidence:.2f}%")

# MAIN----------------------
if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python predict.py <image_path>")
        sys.exit(1)

    img_path = sys.argv[1]
    predict_character(img_path)
```

Output:

```
Model loaded successfully!
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 119ms/step
Predicted Character: N_caps
Confidence: 97.44%
```

```
Model loaded successfully!
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 163ms/step
Predicted Character: z
Confidence: 75.24%
```

```
Model loaded successfully!
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 101ms/step
Predicted Character: B_caps
Confidence: 96.52%
```

1. To achieve more accurate predictions, the input images must be clean, well-preprocessed, and consistently scaled so the model can correctly extract relevant visual features.
2. Increasing training samples and applying stronger data augmentation further enhances the model's generalization ability, resulting in more stable and reliable outputs.

```
Model loaded successfully!
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 104ms/step
Predicted Character: z
Confidence: 75.24%
```

1. Misclassification occurred because the visual features of the input (such as shape, stroke style, or noise) did not closely match the patterns learned during training, which is common in multi-class character recognition where fine-grained variations affect the feature extraction process.
2. To address this, increasing training samples, applying stronger data augmentation, and improving preprocessing (smoothing, resizing consistency, noise removal) can enhance generalization and provide cleaner inputs, ultimately reducing such prediction errors.

6. future enhancement

- Expand the dataset with diverse handwriting styles to improve generalization.
- Apply advanced preprocessing (denoising, stroke enhancement) for cleaner input.
- Use deeper CNN or hybrid CNN-Transformer models for stronger feature extraction.
- Add data augmentation (rotation, skew, blur) to make the model robust.
- Implement learning-rate scheduling, early stopping, and regularization to reduce overfitting.
- Use ensemble predictions to increase reliability and reduce misclassification.

6. Conclusion

In this project, I built a character recognition model using a CNN–ANN approach and successfully trained it on 62 different classes. The model gives good accuracy and works well for recognizing most characters, but a few mistakes still happen due to limited data and visual variations. With more training images, better preprocessing, and some tuning, the model can become even more accurate and reliable in the future.